
Teste Funcional e Classes de Equivalência

Da regra de negócio ao comportamento observável



Prof. Dr. João Choma

Teste e Qualidade de Software

github.com/JoaoChoma

Semana 04



Como projetar testes de caixa-preta, selecionar entradas representativas e registrar resultados reproduzíveis.



Ao final, o estudante deverá ser capaz de:

- explicar a perspectiva externa do teste funcional;
- relacionar requisitos, cenários e casos de teste;
- reconhecer classes de falhas que o teste funcional pode revelar;
- dividir um domínio de entrada em classes de equivalência;
- escolher representantes válidos e inválidos;
- combinar equivalência com análise de valores-limite;
- distinguir teste funcional de teste estrutural.



Plano, casos, roteiro, evidências, relatório e matriz de rastreabilidade tornam o teste **planejado, repetível e verificável**.



- Artefato — Pergunta principal
- Plano de teste — O que, como, quando e com quais recursos testar?
- Caso de teste — Qual situação específica verificar?
- Roteiro — Em que sequência executar as ações?
- Registro de execução — O que aconteceu na execução?
- Relatório de teste — Qual é a situação geral e quais riscos permanecem?
- Matriz de rastreabilidade — Quais requisitos estão cobertos?



Um caso de teste deve informar, no mínimo:

- identificador e requisito relacionado;
- condições;
- dados de entrada;
- passos ou ação;
- resultado esperado;
- resultado obtido e status, após a execução;
- evidências relevantes.

Dica: o resultado esperado precisa ser observável. "Funcionar corretamente" não oferece um critério claro de aprovação.



Requisito: o login deve permitir acesso com credenciais ativas e válidas.

- Campo — Conteúdo
- ID — CT-LOGIN-01
- Precondição — Conta ana@exemplo.com ativa
- Entrada — e-mail válido e senha correta
- Ação — selecionar **Entrar**
- Esperado — abrir a página inicial e criar uma sessão autenticada

Observe: ambiente, versão e identidade dos dados devem ser registrados para que outra pessoa consiga repetir o teste.



No teste **caixa-preta**, selecionamos entradas e observamos saídas sem usar a estrutura interna do código para definir os casos.

- a referência principal é o comportamento especificado;
- os testes exercitam interfaces e regras de negócio;
- o foco está no que o usuário ou outro sistema consegue observar;
- a implementação pode mudar sem invalidar o objetivo do teste.



O teste funcional avalia se cada função produz o comportamento esperado nas condições relevantes.

```
precondição + entrada + ação
      ↓
    sistema
      ↓
saída + mudança de estado + efeitos externos
```

Atenção: não verifique somente a mensagem na tela; banco de dados, evento, e-mail, cobrança e auditoria também podem ser resultados observáveis.



Um **oráculo de teste** é a fonte usada para determinar o resultado esperado:

- requisito ou regra de negócio;
- exemplo de aceitação;
- cálculo independente;
- contrato de API;
- legislação ou política aplicável;
- comportamento de uma versão confiável.

Dica: requisitos ambíguos geram oráculos ambíguos. Resolva a dúvida antes de transformar uma suposição em teste.



- funções ausentes ou incorretas;
- validações de entrada inadequadas;
- erros de interface e integração;
- leitura ou gravação incorreta de dados;
- comportamento incorreto sob estados específicos;
- problemas de inicialização, encerramento e recuperação;
- violações observáveis de desempenho ou tempo de resposta.

Uma execução bem-sucedida comprova apenas o cenário e os dados exercitados.



Regra: uma transferência somente pode ser concluída quando o valor é positivo, o saldo é suficiente e a conta de destino está ativa.

- Cenário — Entrada essencial — Resultado esperado
- fluxo válido — R\$ 100; saldo R\$ 500; destino ativo — concluir uma vez
- valor inválido — R\$ 0 — rejeitar antes de movimentar saldo
- saldo insuficiente — R\$ 600; saldo R\$ 500 — rejeitar sem débito
- destino inativo — R\$ 100; destino inativo — rejeitar e explicar o motivo

Observe: valide também saldo final, histórico, comprovante e ausência de duplicidade.



Um teste não é apenas um conjunto de valores. Considere:

- perfil e permissões do usuário;
- estado anterior dos registros;
- sequência das ações;
- repetição, cancelamento e retorno;
- dependências externas;
- fuso horário, moeda, idioma e dispositivo;
- interrupções como perda de conexão.

Dica: anote as precondições que realmente alteram o resultado; detalhes irrelevantes tornam o caso frágil e difícil de manter.



As perspectivas são complementares:

- Caixa-preta — Caixa-branca
- deriva testes do comportamento — deriva testes da estrutura interna
- observa entradas e saídas — examina decisões, caminhos e fluxo de dados
- encontra regras ausentes ou incorretas — encontra trechos e condições não exercitados
- permanece estável diante de refatorações — acompanha mudanças na implementação

Alta cobertura de código não prova que a regra de negócio está correta.



- Contexto — Exemplos — Uso típico
- Java — JUnit, TestNG — unidade, integração e testes funcionais programados
- Python — pytest — testes programados e parametrizados
- .NET — NUnit — testes automatizados em aplicações .NET
- JavaScript — Jest — lógica JavaScript e aplicações web
- PHP — PHPUnit — testes automatizados em PHP
- BDD — Cucumber — cenários em Gherkin ligados à automação
- Navegador — Selenium — interação com interfaces web
- Automação genérica — Robot Framework — fluxos funcionais e de aceitação



JUnit ou pytest podem executar um teste de unidade, integração ou funcional. O que define o teste é:

- o objetivo;
- o objeto sob teste;
- as dependências reais ou substituídas;
- a interface utilizada;
- o tipo de evidência produzida.

Dica: escolha a camada mais baixa que consiga demonstrar a regra com confiança; deixe a interface gráfica para fluxos realmente críticos.



Regra: idade deve ser um número inteiro de 18 a 65, inclusive.

```
POST /pacientes
Content-Type: application/json

{"nome": "Ana", "idade": 18}
```

Esperado: resposta de sucesso, identificador criado e idade 18 persistida.

Observe: status HTTP, corpo, cabeçalhos relevantes, persistência e ausência de efeitos colaterais indevidos.



Uma classe de equivalência reúne valores que, segundo a regra, devem receber o mesmo tratamento do sistema.

Em vez de testar todos os valores, selecionamos representantes de cada classe.

Para a idade de 18 a 65:

- classe inválida inferior: idade menor que 18;
- classe válida: idade de 18 a 65;
- classe inválida superior: idade maior que 65.



- 1 Identifique o domínio da entrada.
- 2 Localize regras, intervalos, formatos e estados permitidos.
- 3 Separe classes válidas e inválidas.
- 4 Confirme que os valores de uma classe devem ter o mesmo tratamento.
- 5 Escolha ao menos um representante por classe.
- 6 Registre a regra que justifica cada classe.

Dica: se dois valores da mesma classe podem produzir respostas diferentes, a partição está ampla demais e precisa ser dividida.



Se uma função classifica um número como negativo, zero ou positivo:

- Classe — Representante — Esperado
- número negativo — -10 — "negativo"
- zero — 0 — "zero"
- número positivo — 5 — "positivo"

Detalhe importante: zero não pertence às classes positiva ou negativa; é uma classe própria porque recebe tratamento diferente.



Regra: aceitar notas de 0 a 10, inclusive.

- Classe — Exemplo — Esperado
- abaixo da faixa — -2 — rejeitar
- faixa válida — 7,5 — aceitar
- acima da faixa — 12 — rejeitar

Antes de executar, esclareça:

- o separador decimal aceito;
- a quantidade máxima de casas decimais;
- o tratamento de vazio, texto, null, espaços e notação científica.



Para um intervalo inclusivo de 18 a 65, os limites mais informativos são:

- Posição — Valor — Esperado
- imediatamente abaixo do mínimo — 17 — rejeitar
- mínimo — 18 — aceitar
- imediatamente acima do mínimo — 19 — aceitar
- imediatamente abaixo do máximo — 64 — aceitar
- máximo — 65 — aceitar
- imediatamente acima do máximo — 66 — rejeitar

Falhas de comparação como $<$ versus $\leq$ aparecem com frequência nas bordas.



Para idade, não considere apenas números dentro e fora do intervalo:

- inteiro válido: 30;
- inteiro abaixo ou acima da faixa: 17, 66;
- decimal: 18.5;
- texto numérico: "18";
- texto não numérico: "dezoito";
- vazio, espaços e null;
- número extremamente grande;
- estrutura inesperada: lista ou objeto.

Observe: validação no navegador não elimina a necessidade de validação na API ou no serviço.



No login, e-mail e senha podem ser válidos ou inválidos:

- E-mail — Senha — Resultado esperado
- válido — válida — autenticar
- válido — inválida — negar
- inválido — válida — negar
- inválido — inválida — negar

Atenção: a mesma mensagem de erro pode evitar revelar qual credencial existe. Teste também bloqueio, tentativas repetidas e normalização de espaços.



Considere uma senha válida com comprimento entre 8 e 64 caracteres.

- testar apenas Senha123 cobre um valor interno da classe válida;
- testar 7, 8, 9, 63, 64 e 65 caracteres cobre as bordas;
- testar somente comprimento não cobre composição, Unicode ou espaços;
- reutilizar a mesma conta pode introduzir dependência entre execuções.

Dica: escolha representantes para uma razão explícita, não porque são convenientes.



- requisito ou regra de negócio identificada;
- condições controladas;
- classes válidas e inválidas cobertas;
- limites inclusivos ou exclusivos confirmados;
- entrada, ação e resultado esperado inequívocos;
- efeitos colaterais verificados;
- dados independentes ou restaurados;
- execução repetível no ambiente informado;
- evidência suficiente para revisão;
- requisito ligado ao caso na matriz de rastreabilidade.



- repetir muitos valores da mesma classe e esquecer classes inteiras;
- testar somente o fluxo feliz;
- confundir mensagem exibida com estado correto do sistema;
- omitir precondições e dados de teste;
- aceitar requisito ambíguo como oráculo;
- automatizar um caso instável antes de entender seu objetivo;
- depender da ordem de execução;
- considerar uma falha de automação como defeito do produto sem investigar.



Regra: um cupom concede 10% de desconto quando está ativo, dentro da validade, ainda não foi usado pelo cliente e o subtotal é de pelo menos R\$ 100.

Em equipe:

- 1 identifique entradas, estados e efeitos observáveis;
- 2 derive classes válidas e inválidas;
- 3 selecione valores-limite para o subtotal;
- 4 escreva quatro casos de teste completos;
- 5 registre quais riscos ainda não foram cobertos.



- Caso — Condição principal — Esperado
- CT-01 — ativo; válido; não usado; subtotal R\$ 100 — desconto de R\$ 10
- CT-02 — subtotal R\$ 99,99 — nenhum desconto
- CT-03 — cupom expirado; subtotal R\$ 150 — rejeitar como expirado
- CT-04 — cupom já usado; subtotal R\$ 150 — rejeitar sem alterar pedido

Detalhes a observar: arredondamento, fuso horário da validade, moeda, concorrência entre duas compras e idempotência ao reenviar a solicitação.



O teste **caixa-branca** examina decisões, caminhos lógicos, laços, fluxo de dados e colaboração entre componentes.

Ele complementa o teste funcional ao revelar estruturas internas que os cenários externos talvez não exercitem.



```
boolean idadeValida(int idade) {  
    return idade >= 18 && idade <= 65;  
}
```

Caixa-preta: escolher 17, 18, 19, 64, 65 e 66 a partir da regra de negócio.

Caixa-branca: confirmar que as condições idade ≥ 18 e idade ≤ 65 assumem resultados verdadeiro e falso durante a suíte.

As duas perspectivas convergem nos limites, mas partem de fontes diferentes.



- teste funcional observa o comportamento externo do sistema;
- requisitos e oráculos sustentam o resultado esperado;
- cenários devem considerar entradas, estado, sequência e efeitos;
- classes de equivalência reduzem o domínio sem perder tratamentos distintos;
- valores-limite verificam as bordas onde defeitos são frequentes;
- ferramentas apoiam o teste, mas não substituem o projeto dos casos;
- testes caixa-preta e caixa-branca fornecem evidências complementares.



- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 7. ed. Porto Alegre: AMGH, 2011.
- SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.
- PFLEEGER, Shari Lawrence. *Engenharia de Software: teoria e prática*. 2. ed. São Paulo: Pearson, 2004.